

This指针

指向成员函数所作用的对象

每个成员函数都有一个**this**指针，你只是看不到它而已。

```
// 你写的代码
class A {
    int x;
    void foo(int v) { x = v; }
};

// 编译器实际生成的（伪代码）
void A_foo(A* const this, int v) {
    this->x = v;
}
```

this指针在大部分情况下可以省略，但在一些情况中发挥重要作用：

1. 解决类的成员变量名与传入参数名的冲突

```
class Person { private: int age;
public: // 参数名也是 age，如果不加 this，编译器会以为你在用参数给自己赋值（age = age）
    void setAge(int age) {
        this->age = age; // this->age 是类的成员，右边的 age 是传进来的参数
    }
};
```

2. 返回对象本身(链式调用)

```
class Car {
public:
    Car& setEngine() {
        cout << "安装引擎";
        return *this; // 返回对象本身
    }
    Car& setWheels() {
        cout << "安装轮子";
        return *this;
    }
}
```

```
};

int main() {
    Car myCar;
    // 只有返回了 *this, 才能一直点下去
    myCar.setEngine().setWheels();
}
```

3. 把“自己”作为参数传给别的函数

```
class Player;

class Game {
public:
    void addPlayer(Player* p) { /* 把玩家加入游戏房间 */ }
};

class Player {
public:
    void joinGame(Game& game) {
        // 玩家主动加入游戏。我要把“我自己”传给 game 对象，怎么传？只能用 this!
        game.addPlayer(this);
    }
};
```

4. 防止自赋值

```
class MyString {
    char* data;
public:
    MyString& operator=(const MyString& other) {
        // 如果对象的地址 和 传进来的对象的地址 一样，说明是 obj = obj;
        if (this == &other) {
            return *this; // 直接返回，防止后续代码把自己的数据误删了
        }

        // 否则执行正常的赋值逻辑...
        return *this;
    }
};
```

静态成员函数中不能使用**this**指针（不具体作用于某个对象）